

Day 54: Cart Total Method

Day 54: Cart Total Method

1. Day Number

Day 54

2. Topic Name

Method with Accumulator

Today you will learn how a method can:

- loop through a list of objects
- collect values from those objects
- keep adding them into a variable
- return the final result

The variable that keeps collecting the total is called an **accumulator**.

3. Connection

You can already add products to a cart.

A cart may contain products like:

```
Laptop    → 50000  
Mouse     → 1000  
Keyboard  → 2000
```

Today you will make the Cart class calculate:

```
50000 + 1000 + 2000 = 53000
```

You will add a method called:

```
calculate_total()
```

The important difference today is that the method should **return** the total instead of only printing it.

4. Important Topics

Loop

A loop lets us visit every product stored in the cart.

Conceptually:

```
for product in cart_items:  
    ...
```

List of Objects

Your cart contains Product objects.

For example:

```
Cart
|
+-- Product object
+-- Product object
+-- Product object
```

Each product may contain attributes such as:

```
name
price
stock
```

So while looping, you can access:

```
product.price
```

Accumulator

An accumulator is a variable that keeps a running total.

For example:

```
total = 0
```

Then:

```
total = 0

add 100
total = 100

add 200
total = 300

add 50
total = 350
```

Here, total is the accumulator.

Return Value

A method can calculate something and send the result back using:

```
return
```

Conceptually:

```
def some_method(self):  
    result = ...  
    return result
```

Then the returned value can be stored:

```
answer = object.some_method()
```

5. Foundational Notes

A. Start the accumulator at zero

When calculating a sum, normally begin with:

```
total = 0
```

Why zero?

Because initially nothing has been added.

B. Visit every object

Suppose the cart contains:

```
Product("Book", 500)  
Product("Pen", 20)  
Product("Bag", 1000)
```

The loop should examine each product one at a time.

```
First product → Book  
Second product → Pen  
Third product → Bag
```

C. Access the object's price

Because each item is a Product object, its price can be accessed through its attribute.

Concept:

```
product.price
```

D. Add price to the accumulator

For every product:

```
total = total + product price
```

A shorter Python form is:

```
total += product_price
```

Both mean the same basic thing.

E. Return after the loop

The total should be returned **after all products have been processed**.

Conceptually:

```
start total

loop:
    add product price

return total
```

Be careful not to return too early.

If return happens inside the loop, the method may stop after the first product.

6. Easy Example

Before applying this idea to objects, consider a simple list of numbers:

```
prices = [100, 200, 50]

total = 0

for price in prices:
    total += price

print(total)
```

Output:

```
350
```

The important pattern is:

```
Initialize
  ↓
Loop
  ↓
Accumulate
  ↓
Use final result
```

With your cart, the pattern will be similar, except the list contains **Product objects instead of numbers**.

For example:

```
items
|
+-- Product → price = 100
+-- Product → price = 200
+-- Product → price = 50
```

So instead of adding the item itself, you use the object's price.

7. Problem Statement

Add a:

```
calculate_total()
```

method to the Cart class.

The method should:

1. Start a total at 0.
2. Loop through every Product object in the cart's `items` list.
3. Get each product's price.
4. Add the price to the total.
5. Return the final total.

Example cart:

```
Book      ₹500
Pen       ₹20
Notebook  ₹100
```

The method should calculate:

```
500 + 20 + 100
```

and return:

```
620
```

8. Concepts Used

You will use:

- class
- object
- method
- self
- list attribute
- list of objects
- for loop

- object attributes
- accumulator variable
- return
- method call

The central idea is:

```

Cart object
  |
  | items
  v
[Product, Product, Product]
  |
  | loop
  v
read each .price
  |
  v
add into total
  |
  v
return total

```

9. Thought Process

Before writing code, think through the problem.

Step 1: Where are the products?

Inside the cart's:

```
items list
```

Step 2: What information do I need from each product?

Its:

```
price
```

Step 3: How do I examine every product?

Use a:

```
for loop
```

Step 4: Where do I keep the running sum?

Create:

```
total = 0
```

Step 5: What happens for each product?

Add its price:

```
total = total + product price
```

Step 6: What happens after every product has been processed?

Return:

```
total
```

So the overall thinking is:

```
Create total
  |
  v
Loop through items
  |
  v
Read product.price
  |
  v
Add price to total
  |
  v
More products?
 /      \
Yes      No
 |        |
loop    return total
```

10. Pseudocode

```
CLASS Cart

    CONSTRUCTOR
        create empty items list

    METHOD calculate_total

        set total to 0

        FOR each product in items
            add product price to total

        RETURN total
```

Notice that this is only pseudocode, not the complete Python solution.

11. Suggested Solving Approach

Use an **OOP-based approach** because both Product and Cart represent real objects.

Your structure can conceptually look like:

```
Product class
|
+-- name
+-- price
+-- stock

Cart class
|
+-- items
|
+-- add_product()
|
+-- remove_product()
|
+-- calculate_total()
```

The responsibility of each class is becoming clearer:

```
Product
    stores product information

Cart
    manages products
    adds products
    removes products
    calculates total
```

This is one of the important ideas in OOP: **put related behavior with the object responsible for it.**

12. Easy Edge Cases

Edge Case 1: Empty Cart

Suppose:

```
items = []
```

There are no products.

Your accumulator starts as:

```
total = 0
```


The loop runs zero times.

Therefore the method should naturally return:

0

You do not necessarily need a special condition for this.

Edge Case 2: Product Price is 0

Suppose:

Product A → 100
Product B → 0
Product C → 50

Calculation:

$100 + 0 + 50$

Result:

150

A zero-price product should not cause an error.

Small Mental Test

Imagine:

Cart items:

Apple → 50
Milk → 60
Bread → 40

Trace the accumulator:

Starting total = 0

Apple:
 $0 + 50 = 50$

Milk:
 $50 + 60 = 110$

Bread:
 $110 + 40 = 150$

Final result:

150

13. Expected Input

You may create products such as:

```
Product 1:  
name  = "Book"  
price = 500  
  
Product 2:  
name  = "Pen"  
price = 20  
  
Product 3:  
name  = "Bag"  
price = 1000
```

Add them to the cart.

Then call:

```
cart.calculate_total()
```

There does not need to be keyboard input for this exercise unless you want to extend it later.

14. Expected Output

For:

```
Book = 500  
Pen  = 20  
Bag  = 1000
```

Expected total:

```
1520
```

You could eventually display something like:

```
Cart Total: 1520
```

For an empty cart:

```
Cart Total: 0
```

15. Hint Only

Inside `calculate_total()`:

- ```
1. Create a variable starting at 0.
2. Loop through self.items.
```

3. Each item is a Product object.
4. Read the item's price attribute.
5. Add that price to your accumulator.
6. After the loop finishes, return the accumulator.

Think about this skeleton without filling in the missing parts:

```
def calculate_total(self):
 total = ____

 for product in ____:
 total = total + ____

 return ____
```

The key question is:

**What attribute of each product object contains the value you need to add?**